

Conversación con Gemini

Cómo debo usar el `df.where` en python?

El método `df.where()` de **pandas** es una herramienta poderosísima para filtrar o reemplazar datos basándose en una condición lógica. A diferencia de un filtro tradicional que "elimina" filas, `where` mantiene la forma original de tu DataFrame pero cambia los valores que **no** cumplen la condición.

Aquí te explico cómo dominarlo de forma sencilla.

1. La lógica fundamental

La clave para no confundirse con `df.where` es recordar esto:

"Mantén los valores donde la condición sea VERDADERA; reemplaza donde sea FALSA."

Es el comportamiento opuesto al `if` tradicional de Python o al `where` de SQL.

La sintaxis básica:

Python

```
df.where(condicion, reemplazo, inplace=False)
```

- **condicion:** Una serie de booleanos (True/False).
- **reemplazo:** Qué poner donde la condición sea False. Si no pones nada, pandas pone NaN.

2. Ejemplos prácticos

Supongamos que tenemos este DataFrame de temperaturas:

	Ciudad	Temp
0	Madrid	25
1	Berlín	12
2	Roma	30

Escenario A: Reemplazar con NaN (Default)

Queremos ver solo las temperaturas mayores a 20. Lo que no cumpla, se vuelve "nulo".

Python

```
import pandas as pd

df = pd.DataFrame({'Ciudad': ['Madrid', 'Berlín', 'Roma'], 'Temp': [25, 12, 30]})

# Mantener valores > 20, lo demás NaN
resultado = df['Temp'].where(df['Temp'] > 20)
```

Resultado: 25, NaN, 30

Escenario B: Reemplazar con un valor específico

Si queremos que las temperaturas bajas se marquen como "Frío":

Python

```
# Si la temp NO es > 20, poner 'Frío'
df['Estado'] = df['Temp'].where(df['Temp'] > 20, other='Frío')
```

3. df.where vs np.where (NumPy)

Es muy común confundirlos, pero funcionan de forma distinta:

Característica	df.where (Pandas)	np.where (NumPy)
Lógica	Si es Falso , reemplaza.	(condicion, si_True, si_False)
Resultado	Mantiene la estructura de Pandas.	Devuelve un Array o índices.
Uso ideal	Limpieza rápida de DataFrames.	Lógica condicional compleja o rápida.

4. Diferencia con el filtrado clásico

Es importante saber cuándo usar cada uno:

- **Filtrado** (`df[df['A'] > 5]`): Elimina las filas que no cumplen. El DataFrame se hace más pequeño.
- **Where** (`df.where(df['A'] > 5)`): Mantiene todas las filas. Solo "esconde" o cambia los valores que no cumplen. Es ideal cuando necesitas mantener la alineación de los datos o el mismo número de filas para cálculos posteriores.

Un truco extra:

Puedes encadenar `where` con `.fillna()` o usarlo para validar datos en una columna específica sin alterar las demás.

¿Estás intentando limpiar un dataset específico o solo explorando la función para un análisis general?

Si quiero hacer una operación únicamente cuando se cumpla una condición en la fila se ocupa el `where`?

La respuesta corta es: **No es la opción más eficiente.**

Aunque puedes usar `df.where()` para eso, este método está diseñado principalmente para **reemplazar o "tapar"** datos (como poner un parche). Si lo que buscas es aplicar una operación matemática o de transformación solo a un subconjunto de filas, tienes mejores herramientas.

Aquí te presento las tres formas correctas de hacerlo, dependiendo de qué tan compleja sea tu operación:

1. La opción ganadora: `df.loc`

Esta es la forma estándar y más eficiente en Pandas. Te permite seleccionar las filas que cumplen la condición y actuar **solo** sobre una columna específica.

Python

Ejemplo: Solo a los empleados con 'Ventas' > 100, aplicar un bono del 10%

```
df.loc[df['Ventas'] > 100, 'Sueldo'] = df['Sueldo'] * 1.10
```

- **Pros:** Es extremadamente rápido y modifica el DataFrame original directamente.
- **Contras:** Solo sirve para operaciones que se pueden vectorizar (matemáticas simples).

2. La alternativa flexible: `np.where` (de NumPy)

Si quieres crear una **nueva columna** basada en una condición (tipo "si pasa esto, haz A; si no, haz B"), esta es la mejor opción. Es mucho más intuitiva que el `where` de Pandas.

Python

```
import numpy as np
```

```
# Sintaxis: np.where(condicion, valor_si_true, valor_si_false)
```

```
df['Categoria'] = np.where(df['Edad'] >= 18, 'Adulto', 'Menor')
```

- **Pros:** Muy legible y permite definir qué pasa en ambos casos (cuando se cumple y cuando no).

3. Para operaciones complejas: apply con axis=1

Si la operación que quieres hacer es una función muy compleja que no se puede escribir con una simple multiplicación (por ejemplo, algo que involucre lógica de varias columnas), usa apply.

Python

```
def mi_logica_compleja(fila):
```

```
    if fila['Puntaje'] > 80 and fila['Asistencia'] > 0.9:
```

```
        return fila['Sueldo'] * 1.25
```

```
    return fila['Sueldo']
```

```
df['Sueldo'] = df.apply(mi_logica_compleja, axis=1)
```

- **Pros:** Puedes programar cualquier lógica imaginable.
- **Contras:** Es mucho más lento en DataFrames muy grandes (millones de filas).

Resumen: ¿Cuándo usar cada uno?

Si quieres...

Usa...

Cambiar un valor existente basado en condición

df.loc[condicion, 'columna']

Crear columna nueva con lógica "Si / Si no"

np.where()

Si quieres...

Usa...

Tapar o esconder datos que no cumplen algo

`df.where()`

Lógica muy compleja por cada fila

`df.apply(..., axis=1)`

¿Qué tipo de operación estás intentando realizar? Si me das un ejemplo del cálculo, puedo decirte exactamente cuál de estas opciones te conviene más.

Si tengo tres columnas de datos que representan las coordenadas en x, y, z respectivamente, cómo puedo graficar en 3d en python esos puntos unidos por líneas?

Para graficar en 3D en Python, la librería estándar es **Matplotlib** a través de su módulo `mpl_toolkits.mplot3d`.

Para unir los puntos con líneas, el método que necesitas es `plot` (en lugar de `scatter`, que solo dibuja los puntos sueltos).

Aquí tienes el código paso a paso para lograrlo:

Código base para graficar líneas en 3D

Python

```
import matplotlib.pyplot as plt
```

```
import pandas as pd
```

```
# Supongamos que tu DataFrame es 'df'
```

```
# df = pd.DataFrame({'x': [1, 2, 3], 'y': [4, 5, 2], 'z': [10, 12, 11]})
```

```
fig = plt.figure(figsize=(10, 7))
```

```
ax = fig.add_subplot(111, projection='3d')
```

```
# Graficar los puntos unidos por líneas
```

```
ax.plot(df['x'], df['y'], df['z'], label='Trayectoria', color='blue', marker='o')
```

```
# Etiquetas de los ejes
```

```
ax.set_xlabel('Eje X')
ax.set_ylabel('Eje Y')
ax.set_zlabel('Eje Z')
ax.set_title('Gráfico de Líneas en 3D')
ax.legend()

plt.show()
```

Detalles importantes que debes conocer:

1. `projection='3d'`: Es el paso más crítico. Sin esto, Matplotlib intentará crear un gráfico en 2D y te dará un error al intentar pasarle la coordenada z.
2. `ax.plot` **vs** `ax.scatter`:
 - Si usas `ax.plot(x, y, z)`, Python conecta los puntos en el orden en que aparecen en el DataFrame.
 - Si usas `marker='o'`, verás tanto la línea como el punto marcado; si lo quitas, solo verás la línea continua.
3. **Interactividad**: Si estás usando un Jupyter Notebook o VS Code, añade la línea `%matplotlib widget` o `%matplotlib notebook` al principio del script para poder rotar el gráfico con el mouse. De lo contrario, verás una imagen estática.

¿Y si quieres algo más profesional/interactivo?

Si necesitas que el usuario pueda hacer zoom y ver los valores exactos al pasar el cursor, te recomiendo usar **Plotly**, que es mucho más moderno para la web:

Python

```
import plotly.express as px
```

```
fig = px.line_3d(df, x='x', y='y', z='z', title='Línea 3D Interactiva')
```

```
fig.show()
```

¿Estos puntos representan alguna trayectoria física o son datos estadísticos dispersos?
Dependiendo de eso, podrías querer ajustar el orden de los datos antes de graficar.

Puedo graficar segmentos de mi df de diferente color?

¡Totalmente! En **Matplotlib**, la función plot estándar dibuja una sola línea continua. Si quieres que diferentes segmentos tengan colores distintos, tienes dos caminos principales dependiendo de cómo estén estructurados tus datos.

Aquí te muestro las dos mejores formas de hacerlo:

1. La opción manual (Ideal para pocos segmentos)

Si sabes exactamente dónde termina un color y empieza el otro (por ejemplo, los primeros 50 puntos de un color y los siguientes 50 de otro), simplemente haz múltiples llamadas a `ax.plot()`.

Python

```
fig = plt.figure()
```

```
ax = fig.add_subplot(111, projection='3d')
```

```
# Segmento 1: del índice 0 al 10
```

```
ax.plot(df['x'][:11], df['y'][:11], df['z'][:11], color='red', label='Fase 1')
```

```
# Segmento 2: del índice 10 al 20 (fíjate que repetimos el 10 para que no haya hueco)
```

```
ax.plot(df['x'][10:21], df['y'][10:21], df['z'][10:21], color='blue', label='Fase 2')
```

```
ax.legend()
```

```
plt.show()
```

2. La opción con "Groupby" (La más profesional)

Si en tu DataFrame tienes una columna que indica a qué categoría pertenece cada punto (por ejemplo, una columna llamada Estado o Cluster), puedes iterar sobre ella.

Python

```
fig = plt.figure()
```

```
ax = fig.add_subplot(111, projection='3d')
```

```
# Supongamos que la columna 'Segmento' tiene valores como 'A', 'B', 'C'
```

```
colores = {'A': 'red', 'B': 'green', 'C': 'blue'}
```

```
for nombre_grupo, datos in df.groupby('Segmento'):
```

```
    ax.plot(datos['x'], datos['y'], datos['z'],  
            color=colores[nombre_grupo],  
            label=nombre_grupo,  
            marker='o')
```

```
ax.legend()
```

```
plt.show()
```

3. El "Hack" para gradientes (LineCollection)

Si quieres que el color cambie punto a punto (como un gradiente basado en el tiempo o en la altura), Matplotlib es un poco más rígido. Sin embargo, con **Plotly** esto es trivial y se ve increíble:

Python

```
import plotly.express as px
```

```
# 'color' puede ser una columna numérica (gradiente) o categórica
```

```
fig = px.line_3d(df, x='x', y='y', z='z', color='z')
```

```
fig.show()
```

Un detalle técnico importante:

Cuando grafiques segmentos por separado en Matplotlib, asegúrate de que el **último punto del segmento A** sea también el **primer punto del segmento B**. Si no lo haces, verás un "hueco" o espacio vacío entre los colores en lugar de una línea continua que cambia de color.

¿Tienes una columna específica en tu DataFrame que define estos colores o quieres cambiarlos según el valor de las coordenadas (por ejemplo, "si que sea rojo")?

Qué está mal acá?

```
df_DE_MA = df[df['escenario_marea'] == "Base" & df["escenario_marea"] == "Marea  
alta"].copy()
```

El error principal en tu código es de **lógica booleana** y de **sintaxis de operadores** en Pandas.

Aquí están los tres puntos que están fallando:

1. El error lógico (& vs |)

Estás usando el operador **AND** (&). Tu código le está pidiendo a Pandas: *"Tráeme las filas donde el escenario sea 'Base' Y AL MISMO TIEMPO sea 'Marea alta'"*. Como una celda no puede tener dos valores distintos a la vez, el resultado siempre será un DataFrame vacío.

- **Solución:** Debes usar el operador **OR** (|).

2. Falta de paréntesis

En Pandas, cuando combinas condiciones, **cada condición debe ir encerrada entre paréntesis**. Esto se debe a que los operadores bitwise (&, |) tienen una jerarquía de precedencia mayor que los operadores de comparación (==).

3. Solución corregida

Python

Opción con OR (|) y paréntesis

```
df_DE_MA = df[(df['escenario_marea'] == "Base") | (df["escenario_marea"] == "Marea alta")].copy()
```

Una forma más elegante: .isin()

Si tienes que filtrar por varios valores en una misma columna, es mucho más limpio y menos propenso a errores usar el método `.isin()`:

Python

Esta es la mejor práctica

```
df_DE_MA = df[df['escenario_marea'].isin(["Base", "Marea alta"])].copy()
```

¿Por qué esto es mejor?

- **Legibilidad:** Es más fácil de leer.
- **Escalabilidad:** Si mañana quieres agregar "Marea baja", solo lo sumas a la lista.
- **Evitas paréntesis:** No tienes que preocuparte por la jerarquía de los operadores lógicos.

¿Esto era para separar los datos que luego vas a graficar con diferentes colores en el gráfico 3D? Si es así, ¡este filtro es el paso previo perfecto!

Hay algún método numérico de iteración que sea eficiente en python?

Si te refieres a realizar cálculos numéricos repetitivos sobre un DataFrame o arreglos (donde los bucles for tradicionales son desesperadamente lentos), la respuesta no es un "método" único, sino una **jerarquía de eficiencia**.

En Python, la eficiencia numérica se basa en evitar el "overhead" del intérprete. Aquí tienes las opciones ordenadas de mejor a peor:

1. Vectorización con NumPy / Pandas (El estándar de oro)

Antes de intentar iterar, debes intentar **vectorizar**. Esto significa aplicar la operación a todo el arreglo a la vez. Los cálculos se ejecutan en C, no en Python.

- **Lento:** Usar un for para sumar dos columnas.
- **Eficiente:** `df['C'] = df['A'] + df['B']`.

2. Numba: El "truco de magia" JIT

Si tienes una función matemática con bucles complejos que no puedes vectorizar fácilmente (por ejemplo, una simulación donde el paso depende del), usa **Numba**.

Numba compila tu código Python a código de máquina en tiempo real (Just-In-Time).

Python

```
from numba import jit
```

```
import numpy as np
```

```
@jit(nopython=True) # Esto acelera el bucle a velocidades de C/C++
```

```
def iteracion_eficiente(arreglo):
```

```
    resultado = np.zeros_like(arreglo)
```

```
    for i in range(1, len(arreglo)):
```

```
        # Ejemplo de cálculo que depende del valor anterior
```

```
        resultado[i] = arreglo[i] * 0.5 + resultado[i-1] * 0.2
```

```
    return resultado
```

3. Métodos iterativos de Scipy

Si te refieres a **métodos numéricos clásicos** (como encontrar raíces, resolver sistemas de ecuaciones o integrales), no los programes a mano. Usa `scipy.optimize` o `scipy.integrate`.

- **Para optimización/raíces:** `scipy.optimize.newton` o `scipy.optimize.minimize`.
- **Para sistemas lineales:** `scipy.sparse.linalg.cg` (Gradiente Conjugado), que es mucho más eficiente para matrices grandes que una inversión directa.

4. `np.frompyfunc` o `np.vectorize`

Si tienes una función de Python puro y quieres aplicarla a un array, `np.vectorize` la hace más cómoda de usar, aunque **no la hace más rápida** (sigue siendo un bucle por debajo). Úsalo solo por conveniencia estética.

Resumen de rendimiento

Método	Velocidad	Cuándo usarlo
Vectorización	Ultra rápida	Operaciones matemáticas simples.
Numba (@jit)	Ultra rápida	Bucles complejos, simulaciones físicas.
Cython	Máxima	Si necesitas control total de memoria (más complejo).
apply (Pandas)	Lenta	Solo si no hay otra opción.
Bucle for	Muy lenta	Casi nunca para datos numéricos.

¿Cuál es tu caso específico? ¿Estás resolviendo una ecuación diferencial, buscando una raíz, o simplemente procesando millones de filas con una fórmula?

Mediante un balance de energía, de la que depende la pérdida de carga por fricción (f), y que a su vez depende de la velocidad del fluido, tengo que ir jugando con el valor de la velocidad a la que va el flujo en la que se cumpla mi balance de energía. Debido a que determinar f es complejo y no trivial, no puedo despejar la velocidad de manera trivial

Lo que describes es un problema clásico de **ingeniería de procesos**: una ecuación implícita donde la velocidad (v) aparece en ambos lados del balance (directamente y "escondida" dentro del factor de fricción a través del número de Reynolds).

Para este caso, **no uses bucles for manuales**. El método numérico más eficiente y estándar en Python es el **Método de Newton-Raphson** o el **Método de Brent**, ambos disponibles en `scipy.optimize`.

Aquí tienes cómo estructurarlo:

1. Define tu función "Residuo"

En métodos numéricos, reescribimos la ecuación como $f(v) = 0$. Es decir:

2. Ejemplo de implementación eficiente

Python

```
import numpy as np
```

```
from scipy.optimize import brentq # Método de Brent (robusto y rápido)
```

```
# Supongamos estos datos
```

```
D = 0.1    # Diámetro
```

```
rho = 1000 # Densidad
```

```
mu = 0.001 # Viscosidad
```

```
epsilon = 0.000045 # Rugosidad absoluta
```

```
def calcular_f_moody(v, D, rho, mu, epsilon):
```

```
    """Calcula f usando la ecuación de Colebrook-White (o una aproximación)"""
```

```
    Re = (rho * v * D) / mu
```

```
    if Re < 2300:
```

```
        return 64 / Re
```

```
    # Aproximación de Haaland (más rápida que iterar Colebrook dentro de otra iteración)
```

```
    a = (epsilon / (3.7 * D))**1.11
```

```
    b = 6.9 / Re
```

```
    f = (-1.8 * np.log10(a + b))**-2
```

```
    return f
```

```
def balance_energia(v, *args):
```

```
    # h_total = h_estatica + h_friccion(v)
```

```
    # Queremos que: Energia_disponible - h_total(v) = 0
```

```
    h_disponible = args[0]
```

```
    L = args[1]
```

```
    g = 9.81
```

```

f = calcular_f_moody(v, D, rho, mu, epsilon)

h_friccion = f * (L / D) * (v**2 / (2 * g))

return h_disponible - h_friccion - (v**2 / (2 * g))

# --- RESOLVER ---

L_tuberia = 100

H_disp = 10 # metros de columna de agua disponibles

# Buscamos la raíz entre v=0.01 y v=10 m/s

velocidad_final = brentq(balance_energia, 0.01, 10, args=(H_disp, L_tuberia))

print(f"La velocidad equilibrada es: {velocidad_final:.4f} m/s")

```

¿Por qué este método es el mejor?

1. **Convergencia Garantizada:** brentq es un método híbrido que combina bisección (seguro) con interpolación cuadrática (rápido). Si la raíz está en el intervalo que le das, la encontrará.
2. **Eficiencia de Scipy:** Estas funciones están escritas en C optimizado. Son miles de veces más rápidas que ir incrementando la velocidad en un for de en .
3. **Manejo de Colebrook-White:** Como determinar ya requiere su propia iteración, tener una "iteración dentro de otra iteración" puede ser lento. Usar la **ecuación de Haaland** (como puse en el ejemplo) para calcular directamente es un truco de ingeniería común para acelerar el cálculo sin perder precisión significativa.

Recomendación Pro:

Si tienes que hacer este cálculo para **muchas tuberías a la vez** (por ejemplo, en un DataFrame con 10,000 tramos), entonces no uses brentq. En ese caso, usa la **vectorización** con la ecuación de Swamee-Jain (que es una forma explícita de calcular la pérdida de carga sin iterar) para obtener un resultado instantáneo en todo el DataFrame.

¿Necesitas resolver esto para una sola tubería o para una red completa de datos en un DataFrame?

Entonces al ser un método implícito, cuál me conviene más? Cuál de los dos métodos? o hay alguna literatura que me permita encontrar la diferencia de las dos?

Para resolver ecuaciones implícitas en ingeniería, como tu balance de energía donde la velocidad está "atrapada" en el factor de fricción, la elección depende totalmente de la **estabilidad** de tu función y de qué tan cerca estés de una solución inicial.

Entre los dos grandes candidatos (Newton-Raphson y Brent), aquí tienes la comparativa técnica para que decidas:

1. Newton-Raphson (scipy.optimize.newton)

Es el método de "ataque rápido". Usa la derivada de la función para saltar hacia la raíz.

- **Cuándo usarlo:** Si tienes una estimación inicial muy buena (un "guess") y la función es suave.
- **Ventaja:** Es extremadamente rápido (convergencia cuadrática).
- **Desventaja:** Si la curva de tu balance de energía tiene pendientes muy pronunciadas o tu valor inicial es malo, el método puede "salir disparado" al infinito y **no converger**.
- **Literatura:** Cualquier libro de *Análisis Numérico* (ej. **Chapra & Canale**). Se basa en la Serie de Taylor.

2. Método de Brent (scipy.optimize.brentq) - El Recomendado

Es un método "híbrido". Combina bisección, secante e interpolación cuadrática inversa.

- **Cuándo usarlo:** En problemas de ingeniería donde no quieres fallos catastróficos de convergencia. **Es el que te conviene más para tu balance de energía.**
- **Ventaja:** Es **robusto por definición**. Si le das un intervalo (un valor de velocidad donde el residuo es positivo y otro donde es negativo), está garantizado matemáticamente que encontrará la raíz.
- **Desventaja:** Necesitas definir un rango (por ejemplo, entre y m/s).
- **Literatura:** *Numerical Methods for Scientists and Engineers* (Richard Hamming).

Cuadro Comparativo: Brent vs Newton

Característica	Newton-Raphson	Método de Brent
Velocidad	La más alta (si converge).	Muy alta.

Característica	Newton-Raphson	Método de Brent
Fiabilidad	Puede fallar (divergir).	Garantizada (si hay cambio de signo).
Requerimiento	Un valor inicial ().	Un intervalo .
Derivadas	Necesita (o la estima).	No necesita derivadas.

¿Dónde leer más? (Literatura recomendada)

1. **"Métodos Numéricos para Ingenieros" (Steven Chapra):** Es la biblia en español para esto. Busca el capítulo de "Raíces de Ecuaciones". Explica perfectamente por qué Colebrook-White es un problema y cómo Newton vs. Bisección (base de Brent) se comportan.
2. **Documentación de Scipy:** Busca `scipy.optimize`. Ahí explican que `brentq` es generalmente considerado el mejor algoritmo de búsqueda de raíces en una dimensión.

Mi consejo profesional:

Para un balance de energía con pérdida de carga: **Usa el Método de Brent** (`brentq`). La física del flujo (especialmente en la transición de laminar a turbulento) puede crear discontinuidades o pendientes extrañas en la derivada que harían que Newton falle. Brent manejará esos "saltos" sin inmutarse.

¿Ya tienes definidos los límites de velocidad mínima y máxima para tu sistema (por ejemplo, de a m/s)?

Me puedes explicar el método Brent? Ya que tengo el rango de valores impuesto en el problema... que no entendí muy bien cómo tengo que plantear la ecuación

El **Método de Brent** es genial porque es "inteligente": intenta ir rápido como un corredor de carreras, pero si ve que el terreno es difícil, se vuelve precavido como un senderista.

Para usarlo, no le das una ecuación para que la despeje, sino una **función de error** (o residuo) que el algoritmo intentará llevar a **cero**.

1. Cómo plantear la ecuación (La función "Residuo")

Imagina que tu balance de energía es:

Como no puedes despejar , creas una función donde pasas todo a un solo lado. Esta es la función que le darás a Python:

- Si eliges una **muy pequeña**, la pérdida por fricción será casi cero y será un número **positivo** (sobra energía).
 - Si eliges una **muy grande**, la fricción será enorme y será un número **negativo** (falta energía).
 - **El Método de Brent buscará el valor exacto de donde .**
-

2. ¿Cómo funciona Brent por dentro?

Brent combina tres estrategias y decide cuál usar en cada paso:

1. **Bisección:** Divide el intervalo a la mitad. Es lento pero **infalible**. Si el método se vuelve loco, siempre vuelve a esto para asegurar que no se pierde.
2. **Método de la Secante:** Traza una línea recta entre dos puntos para adivinar dónde está el cero. Es mucho más rápido que la bisección.
3. **Interpolación Cuadrática Inversa:** Es la "magia". Mira los últimos tres puntos calculados, ajusta una parábola y salta directamente a donde la parábola cruza el cero. Es increíblemente veloz.

La lógica es: Intenta usar la parábola; si no funciona, intenta la secante; y si nada de eso parece seguro, usa bisección.

3. Ejemplo de código real (Estructura de tu problema)

Aquí tienes cómo deberías escribirlo en Python. Nota que el método requiere que en los extremos del rango (y), la función tenga **signos opuestos**.

Python

```
from scipy.optimize import brentq
```

```
# 1. Defines la función con la lógica de ingeniería
```

```
def mi_balance_energia(v, parametros):
```

```
    # Supongamos que recibes tus constantes en un diccionario
```

```
    z1, z2 = parametros['z1'], parametros['z2']
```

```
    L, D = parametros['L'], parametros['D']
```



```

# Aquí calculas f (el factor de fricción) que depende de v
f = calcular_f_moody(v, D, ...)

# hf es la pérdida de carga
hf = f * (L / D) * (v**2 / (2 * 9.81))

# ESTO ES LO IMPORTANTE: Retornar la diferencia (Residuo)
# Queremos que: (Energía Potencial) - (Pérdidas) - (Energía Cinética final) = 0
return (z1 - z2) - hf - (v**2 / (2 * 9.81))

# 2. Definir el rango y parámetros
rango_v = [0.001, 15.0] # [v_min, v_max] m/s
mis_datos = {'z1': 100, 'z2': 80, 'L': 500, 'D': 0.2}

# 3. Ejecutar Brent
try:
    v_solucion = brentq(mi_balance_energia, rango_v[0], rango_v[1], args=(mis_datos,))
    print(f"La velocidad que cumple el balance es: {v_solucion:.4f} m/s")
except ValueError:
    print("Error: El signo de la función es igual en ambos extremos del rango.")
    print("Prueba ampliando el rango o revisando la física del problema.")

```

¿Por qué te sirve tener el rango impuesto?

Para que Brent funcione, necesitas cumplir la **Condición de Cambio de Signo**:

- debe ser positivo y negativo (o viceversa).

Si el rango que te dieron es, por ejemplo, , el código evaluará la energía con y con . Si en ambos casos te sobra energía, significa que la velocidad real es mayor a , y Brent te lanzará un error avisándote.

Un tip extra de ingeniero: Si el cálculo de (fricción) te está dando problemas porque la ecuación de Colebrook es difícil, usa la **fórmula de Swamee-Jain** dentro de la función; es explícita, muy precisa para régimen turbulento y hará que tu código vuele.

Entonces tengo que definir una función que me retorne el "residuo"

Exactamente. Ese es el concepto fundamental en el análisis numérico: **convertir un problema de igualdad en un problema de búsqueda de raíces.**

En lugar de intentar que Python "despeje" la variable (cosa que no puede hacer con ecuaciones implícitas), tú construyes una función que mide **qué tan equivocado estás.**

La lógica del Residuo

Si tu balance es , tu residuo es .

- Si el residuo es **10**, significa que el lado izquierdo es mayor; te falta "restar" más energía (necesitas más velocidad/fricción).
- Si el residuo es **-5**, te pasaste; el lado derecho es mayor (necesitas menos velocidad).
- Si el residuo es **0**, encontraste la velocidad física real.

Pasos finales para tu código:

1. **Define la función:** Asegúrate de que el primer argumento sea siempre la variable que quieres encontrar (en tu caso, la velocidad v).
2. **Parámetros extra:** Si necesitas usar datos como el diámetro, la viscosidad o la longitud dentro de la función, pásalos a través del argumento args de brentq.
3. **Verifica los extremos:** Antes de correr el método, puedes hacer un print rápido:
 - `print(mi_funcion(v_min, ...))`
 - `print(mi_funcion(v_max, ...))` **Obligatorio:** Uno debe ser positivo y el otro negativo. Si ambos tienen el mismo signo, el balance de energía no se cumple en ninguna parte de ese rango (o hay un error en las unidades).

Ejemplo visual de lo que hará Brent:

Imagina una curva que cruza el eje horizontal. $v_{\min}$ está a la izquierda del cruce y $v_{\max}$ a la derecha. El método de Brent simplemente va "encerrando" ese cruce hasta que el residuo es tan pequeño (por ejemplo,) que para fines de ingeniería es **cero**.

¿Te sientes listo para armar la función o tienes dudas sobre cómo meter la fórmula de la fricción ahí dentro?

Cómo graficar en 2D

Para graficar en 2D en Python, la librería por excelencia es **Matplotlib**. Es la base de casi todas las demás herramientas visuales (como Seaborn o la integración de Pandas).

Aquí te explico cómo usarla, desde lo más básico hasta cómo personalizarla.

1. El esquema básico (Line Plot)

Para hacer una gráfica simple de líneas o puntos, usamos `plt.plot(x, y)`.

Python

```
import matplotlib.pyplot as plt
```

```
# Datos
```

```
x = [1, 2, 3, 4, 5]
```

```
y = [2, 4, 1, 5, 2]
```

```
# Crear la figura
```

```
plt.figure(figsize=(8, 5))
```

```
# Graficar
```

```
plt.plot(x, y, color='blue', linestyle='--', marker='o', label='Mi serie')
```

```
# Personalización
```

```
plt.title('Gráfico en 2D básico')
```

```
plt.xlabel('Eje X (Independiente)')
```

```
plt.ylabel('Eje Y (Dependiente)')
```

```
plt.grid(True) # Cuadrícula
```

```
plt.legend() # Mostrar leyenda
```

```
# Mostrar
```

```
plt.show()
```

2. Tipos de gráficos 2D más comunes

Dependiendo de tus datos, Matplotlib ofrece diferentes funciones:

- **Dispersión (plt.scatter):** Ideal para ver la correlación entre dos variables sin unirlos por líneas.
- **Barras (plt.bar):** Para comparar categorías.
- **Histogramas (plt.hist):** Para ver la distribución de una sola variable.

Python

Ejemplo rápido de Scatter (Dispersión)

```
plt.scatter(x, y, color='red', s=100) # 's' es el tamaño del punto
```

```
plt.show()
```

3. Graficar directamente desde un DataFrame

Si estás trabajando con **Pandas**, no necesitas importar Matplotlib por separado para cosas rápidas. Puedes usar el método `.plot()` del propio DataFrame.

Python

```
import pandas as pd
```

```
df = pd.DataFrame({  
    'Mes': ['Ene', 'Feb', 'Mar', 'Abr'],  
    'Ventas': [100, 150, 120, 200]  
})
```

```
# Graficar directamente
```

```
df.plot(kind='line', x='Mes', y='Ventas', marker='v', color='green')
```

```
plt.show()
```

4. Subplots (Varias gráficas en una imagen)

Si quieres comparar dos gráficos lado a lado, usas `plt.subplots`.

Python

```
# Crear una malla de 1 fila y 2 columnas
```

```
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4))
```

```
# Gráfico 1
```

```
ax1.plot(x, y, 'r-')
```

```
ax1.set_title('Gráfico Rojo')
```

```
# Gráfico 2
```

```
ax2.bar(x, y, color='skyblue')
```

```
ax2.set_title('Gráfico de Barras')
```

```
plt.show()
```

Tips de personalización (El "maquillaje")

Para que tus gráficas pasen de "académicas" a "profesionales", puedes cambiar el **estilo** global al principio de tu código:

Python

```
plt.style.use('ggplot') # Estilo similar a R
```

o también:

```
plt.style.use('seaborn-v0_8') # Estilo más limpio
```

¿Qué quieres graficar exactamente?

¿Son datos de una función matemática, resultados de tus balances de energía o una serie de tiempo? Si me dices el tipo de datos, puedo darte un código de personalización específico (por ejemplo, cómo poner flechas, anotar puntos máximos o cambiar escalas logarítmicas).

Si quiero agregar 2 gráficos con el mismo x, cómo puedo acomodar este código?

```
plt.figure(figsize=(8, 5))
```

```
# Graficar
```

```
plt.plot(df_BC['dist_acum_m'], df_BC['P_BC'], color='blue', linestyle='--', marker='o',  
label='Presión [kPa]')
```

```
plt.plot(df_BC['dist_acum_m'], df_BC['Piezometrico'], color='c', linestyle='--', marker='o',  
label='Línea Piezométrica')
```

```
# Personalización

plt.title('Gráfico en 2D básico')

plt.xlabel('Recorrido [m]')

plt.ylabel('Presión [kPa]')

plt.grid(True) # Cuadrícula

plt.legend() # Mostrar leyenda
```

```
# Mostrar

plt.show()
```

Tu código actual **ya funciona matemáticamente** para poner las dos líneas en el mismo gráfico. Al llamar dos veces a `plt.plot()` antes de `plt.show()`, Matplotlib entiende automáticamente que ambas van en el mismo lienzo.

Sin embargo, para que el código quede estructurado de forma profesional (usando la sintaxis Orientada a Objetos) y sea más fácil de leer o modificar en el futuro, tienes tres formas excelentes de "acomodarlo" dependiendo de cómo quieras que se vea.

Aquí tienes las tres opciones:

Opción 1: Juntos en el mismo gráfico (La forma profesional de tu código)

Esta es tu misma idea, pero usando `fig`, `ax`. Es la mejor práctica en Pandas/Matplotlib porque te da control absoluto sobre ese eje específico.

Python

```
import matplotlib.pyplot as plt
```

```
fig, ax = plt.subplots(figsize=(8, 5))
```

```
# Graficar ambas líneas en el mismo eje (ax)
```

```
ax.plot(df_BC['dist_acum_m'], df_BC['P_BC'], color='blue', linestyle='--', marker='o',
label='Presión [kPa]')
```

```
ax.plot(df_BC['dist_acum_m'], df_BC['Piezometrico'], color='c', linestyle='-', marker='s',
label='Línea Piezométrica')
```

```
# Personalización
```

```
ax.set_title('Perfil Hidráulico a lo largo del recorrido')
```

```
ax.set_xlabel('Recorrido [m]')
```

```
ax.set_ylabel('Presión / Carga') # Nombre genérico si comparten la misma unidad
```

```
ax.grid(True)
```

```
ax.legend()
```

```
plt.show()
```

(Nota: Cambié el marcador de la segunda línea a 's' (cuadrado) y la línea a continua '-' para que visualmente no se confundan si se superponen).

Opción 2: Gráficos separados (Uno arriba del otro)

Si tus líneas se cruzan mucho y hace que la gráfica sea difícil de leer, lo mejor es poner una arriba y otra abajo, **compartiendo el mismo eje X** para que al hacer zoom o mirar los datos, todo esté alineado verticalmente.

Python

```
# Crear 2 filas, 1 columna, y obligar a que compartan el eje X (sharex=True)
```

```
fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(8, 8), sharex=True)
```

```
# --- Gráfico Superior: Presión ---
```

```
ax1.plot(df_BC['dist_acum_m'], df_BC['P_BC'], color='blue', linestyle='--', marker='o',  
label='Presión')
```

```
ax1.set_title('Análisis de Presión y Línea Piezométrica')
```

```
ax1.set_ylabel('Presión [kPa]')
```

```
ax1.grid(True)
```

```
ax1.legend()
```

```
# --- Gráfico Inferior: Piezométrica ---
```

```
ax2.plot(df_BC['dist_acum_m'], df_BC['Piezometrico'], color='c', linestyle='-', marker='s',  
label='Línea Piezométrica')
```

```
ax2.set_xlabel('Recorrido [m]') # El título de X solo se pone en el de abajo
```

```
ax2.set_ylabel('Carga Piezométrica')
ax2.grid(True)
ax2.legend()

# Esto evita que los números de un gráfico choquen con los del otro
plt.tight_layout()
plt.show()
```

Opción 3: Ejes "Gemelos" (Doble eje Y)

Si la Presión está en **kPa** (valores grandes) y la Línea Piezométrica está en **metros** (valores pequeños), graficarlas en el mismo eje "aplastará" a una de las dos. Para solucionarlo, creas un eje secundario a la derecha.

Python

```
fig, ax1 = plt.subplots(figsize=(8, 5))

# Eje Izquierdo (ax1) - Para la Presión
ax1.plot(df_BC['dist_acum_m'], df_BC['P_BC'], color='blue', marker='o', label='Presión [kPa]')
ax1.set_xlabel('Recorrido [m]')
ax1.set_ylabel('Presión [kPa]', color='blue')
ax1.tick_params(axis='y', labelcolor='blue')
ax1.grid(True)

# Eje Derecho (ax2) - Para la Piezométrica
ax2 = ax1.twinx() # Crea un eje Y gemelo que comparte el eje X
ax2.plot(df_BC['dist_acum_m'], df_BC['Piezometrico'], color='c', marker='s', label='Línea Piezométrica [m]')
ax2.set_ylabel('Línea Piezométrica', color='c')
ax2.tick_params(axis='y', labelcolor='c')

ax1.set_title('Perfil de Presiones (Doble Escala)')
```



```
# Para juntar las leyendas de ambos ejes
lines_1, labels_1 = ax1.get_legend_handles_labels()
lines_2, labels_2 = ax2.get_legend_handles_labels()
ax1.legend(lines_1 + lines_2, labels_1 + labels_2, loc='upper right')

plt.show()
```

Cómo puedo añadir una fila nueva al comienzo?

Añadir una fila exactamente al **comienzo** (en el índice 0) requiere un pequeño truco en Pandas, ya que la librería está optimizada para agregar datos al final.

La función `append()` ya fue deprecada, por lo que la forma moderna y correcta de hacerlo es usando `pd.concat()`.

Aquí tienes los dos métodos más usados, desde el más profesional hasta un "hack" rápido.

Método 1: La forma recomendada (`pd.concat`)

Consiste en crear un pequeño DataFrame con tu nueva fila y luego "pegar" tu DataFrame original debajo de él.

Python

```
import pandas as pd
```

```
# Tu DataFrame original
```

```
df = pd.DataFrame({'X': [1, 2], 'Y': [3, 4]})
```

```
# 1. Creas la nueva fila como si fuera un DataFrame de 1 sola fila
```

```
nueva_fila = pd.DataFrame({'X': [99], 'Y': [88]})
```

```
# 2. Los concatenas (poniendo la nueva fila primero)
```

```
df = pd.concat([nueva_fila, df], ignore_index=True)
```

```
print(df)
```

Ventaja: Al usar `ignore_index=True`, Pandas reordena automáticamente los índices de 0 hasta el final, dejando tu nueva fila perfectamente acomodada en el 0.

Método 2: El "hack" con `.loc`

Si por alguna razón no quieres usar `concat`, puedes forzar la inserción usando un índice negativo, y luego reordenar el índice.

Python

```
# Tu DataFrame original
```

```
df = pd.DataFrame({'X': [1, 2], 'Y': [3, 4]})
```

```
# 1. Insertamos la fila en el índice -1 (se agregará al final pero con ese índice)
```

```
df.loc[-1] = [99, 88]
```

```
# 2. Ordenamos el índice para que el -1 pase a estar al principio
```

```
df = df.sort_index()
```

```
# 3. Reseteamos los índices para que vuelvan a empezar desde 0
```

```
df = df.reset_index(drop=True)
```

```
print(df)
```

¿Cuál elegir? El **Método 1** (`pd.concat`) es el estándar de la industria. Es más legible, menos propenso a errores y maneja mejor los casos donde las columnas podrían no estar en el mismo orden exacto (Pandas las alinea por el nombre de la columna automáticamente).

Es posible exportar directamente un gráfico a la carpeta en la que se encuentra el código python en vez de mostrar el gráfico?

¡Sí, absolutamente! De hecho, guardar los gráficos directamente es la práctica más común cuando estás procesando muchos datos o automatizando reportes, ya que no quieres que Python se detenga a mostrarte cada ventana.

Para hacerlo, solo necesitas reemplazar (o poner antes) tu `plt.show()` por la función `plt.savefig()`.

Aquí tienes cómo hacerlo paso a paso:

El código básico

Si solo pones el nombre del archivo, Python lo guardará por defecto en la **misma carpeta** desde donde estás ejecutando tu script.

Python

```
import matplotlib.pyplot as plt
```

```
# 1. Creas tu gráfico normalmente
```

```
x = [1, 2, 3]
```

```
y = [10, 20, 15]
```

```
plt.plot(x, y, color='blue', marker='o')
```

```
plt.title('Mi Gráfico Exportado')
```

```
# 2. Lo guardas en la carpeta actual
```

```
plt.savefig('mi_grafico.png')
```

```
# Opcional: limpiar la memoria si vas a hacer muchos gráficos en un bucle
```

```
plt.close()
```

⚠ La Regla de Oro: **savefig SIEMPRE** antes de **show**

Si en algún momento quieres guardar el gráfico y *además* verlo en pantalla, debes guardar primero.

Python

```
# CORRECTO
```

```
plt.savefig('grafico.png')
```

```
plt.show()
```

```
# INCORRECTO (Guardará una imagen en blanco)
```

```
plt.show()
```

```
plt.savefig('grafico.png')
```

¿Por qué? Porque `plt.show()` "consume" el gráfico. Una vez que cierras la ventana interactiva, Matplotlib limpia el lienzo de dibujo, por lo que si intentas guardar después, guardarás un lienzo vacío.

Tips Profesionales para `savefig`

Puedes pasarle varios parámetros a la función para mejorar drásticamente la calidad de tu imagen exportada:

1. Alta resolución (DPI): Ideal si el gráfico es para un documento de Word, una tesis o un artículo.

Python

```
plt.savefig('grafico_hd.png', dpi=300) # 300 dpi es calidad de impresión
```

2. Eliminar bordes blancos extra: A veces Matplotlib deja mucho margen blanco alrededor. Esto lo recorta perfectamente al borde del gráfico:

Python

```
plt.savefig('grafico_recortado.png', bbox_inches='tight')
```

3. Cambiar el formato: Matplotlib adivina el formato por la extensión que le pongas al archivo. Puedes guardar en formatos vectoriales (que no pierden calidad al hacer zoom).

Python

```
plt.savefig('grafico.pdf') # Excelente para LaTeX o reportes formales
```

```
plt.savefig('grafico.svg') # Excelente para diseño web o Illustrator
```

4. Fondo transparente: Si quieres pegar el gráfico sobre una presentación de PowerPoint con fondo oscuro:

Python

```
plt.savefig('grafico_transparente.png', transparent=True)
```

¡Puedes combinar todos estos parámetros en una sola línea! Por ejemplo:

```
plt.savefig('reporte_final.png', dpi=300, bbox_inches='tight', transparent=True)
```

Si quiero hacer un gráfico de comparación de presiones, cómo puedo agregar esas líneas al mismo gráfico?

Para comparar varias presiones en un mismo gráfico, la lógica es muy sencilla: **solo tienes que llamar a la función `plot` varias veces antes de mostrar el gráfico con `plt.show()` o guardarlo con `plt.savefig()`.**

Matplotlib es lo suficientemente inteligente como para saber que todas esas líneas van en el mismo "lienzo" y ajustará los ejes automáticamente para que todas quepan.

Aquí tienes el código paso a paso usando la estructura profesional (fig, ax) para comparar tres presiones diferentes:

Python

```
import matplotlib.pyplot as plt
```

```
import pandas as pd
```

```
# Supongamos que tienes este DataFrame con tus distancias y 3 presiones distintas
```

```
# df = pd.DataFrame({'distancia': [0, 10, 20, 30], 'P1': [100, 95, 80, 75], 'P2': [100, 90, 70, 60],  
                    'P_teo': [100, 98, 90, 85]})
```

```
fig, ax = plt.subplots(figsize=(10, 6))
```

```
# 1. Agregas todas las líneas que necesites al mismo eje (ax)
```

```
# Es vital usar 'label' para saber cuál es cuál en la leyenda
```

```
ax.plot(df['distancia'], df['P1'], color='blue', linestyle='-', marker='o', label='Presión Real (Tubo A)')
```

```
ax.plot(df['distancia'], df['P2'], color='red', linestyle='--', marker='s', label='Presión Real (Tubo B)')
```

```
ax.plot(df['distancia'], df['P_teo'], color='green', linestyle=':', marker='^', label='Presión Teórica')
```

```
# 2. Personalizas el gráfico
```

```
ax.set_title('Comparativa de Caídas de Presión')
```

```
ax.set_xlabel('Distancia [m]')
```

```
ax.set_ylabel('Presión [kPa]')
```

```
ax.grid(True, linestyle='--', alpha=0.7) # Un grid más suave para que no compita con las líneas
```

```
# 3. Muestras la leyenda (esto unirá los 'labels' con sus respectivos colores)
```

```
ax.legend(loc='best') # 'best' hace que Python busque el hueco vacío para no tapar las líneas
```

```
plt.show()
```

3 claves para que tu gráfica comparativa se vea increíble:

1. **Diferencia los estilos (linestyle):** Si lo vas a imprimir en blanco y negro, los colores no sirven. Usa combinaciones de líneas continuas '-', punteadas ':', y discontinuas '--'.
2. **Diferencia los marcadores (marker):** Usa círculos 'o', cuadrados 's', y triángulos '^' para que cada línea sea inconfundible.
3. **Transparencia (alpha):** Si tienes, por ejemplo, 10 líneas y se amontonan mucho, puedes agregar `alpha=0.5` dentro de tu `plot()`. Esto hará que las líneas sean semitransparentes y podrás ver dónde se superponen varias líneas.

Hay algún marcador más suave? que no interrumpa tanto en lo visual cuando se acumulen puntos en alguna zona?

Sí, cuando tienes mucha densidad de datos, los marcadores clásicos como 'o' o 's' ensucian el gráfico y tapan la tendencia.

Para solucionar el problema de acumulación de puntos, tienes **cuatro técnicas excelentes** en Matplotlib que puedes usar solas o combinadas:

1. Usar marcadores más pequeños (El punto y el píxel)

En lugar del círculo grande ('o'), puedes usar marcadores diseñados específicamente para alta densidad:

- **Punto** (`marker='.'`): Es un círculo mucho más pequeño y discreto.
- **Píxel** (`marker=';'`): Dibuja literalmente un solo píxel. Es el menos invasivo de todos.

Python

```
ax.plot(x, y, marker='.', color='blue') # Un punto fino
```

2. Quitar el "borde" del marcador

Por defecto, Matplotlib le pone un borde de color a los marcadores. Cuando los puntos se amontonan, esos bordes se empalman y crean manchas oscuras. Si le quitas el borde, el color se ve mucho más suave.

Python

```
ax.plot(x, y, marker='o', markeredgecolor='none')
```

3. La regla de oro: Transparencia (alpha)

Esta es la mejor técnica visual para la acumulación. Si haces que los marcadores sean semitransparentes, un solo punto se verá muy suave, pero **donde se acumulen muchos puntos, el color se volverá más intenso**, dándote un efecto visual de "mapa de calor" o densidad.

Python

```
# alpha=0.3 significa 30% de opacidad (70% transparente)
```

```
ax.plot(x, y, marker='o', alpha=0.3)
```

4. El "Truco Pro": Mostrar 1 de cada N puntos (markevery)

Si tu línea tiene 10,000 datos, no necesitas 10,000 marcadores. Puedes decirle a Matplotlib que dibuje la línea continua, pero que solo ponga un marcador cada, por ejemplo, 50 puntos.

Python

```
# Dibuja la línea completa, pero pone un marcador cada 50 datos
```

```
ax.plot(x, y, marker='o', markevery=50)
```

El "Combo Perfecto"

Si juntas todo esto en tu código de presiones, lograrás un gráfico extremadamente limpio y profesional. Aquí te dejo cómo se vería la línea de código ideal para un perfil denso:

Python

```
ax.plot(df['distancia'], df['P_BC'],  
        color='blue',  
        linestyle='-',    # Línea continua suave  
        marker='.',       # Marcador de punto pequeño  
        alpha=0.6,        # Ligeramente transparente  
        markevery=10,     # Un marcador cada 10 puntos (ajusta este número)  
        label='Presión [kPa]')
```

cuál es el figsize más hd posible?

El secreto para lograr la máxima calidad ("HD", 4K o superior) **no está solo en el** figsize, sino en su combinación con el parámetro dpi (Dots Per Inch o Puntos Por Pulgada).

En Matplotlib, la resolución real en píxeles de tu imagen se calcula con esta fórmula:

Resolución en Píxeles = figsize (pulgadas) × dpi

Si solo haces el figsize muy grande (ej. figsize=(50, 30)), los números, las líneas y las letras se verán diminutos en proporción al lienzo. La forma correcta es mantener un figsize proporcionado (como 16:9) y subir el dpi.

Aquí tienes las "recetas" exactas para las resoluciones más buscadas:

1. Calidad Full HD (1080p -> 1920 x 1080 píxeles)

La proporción clásica de pantallas modernas.

Python

Proporción 16:9 con DPI ajustado

```
fig, ax = plt.subplots(figsize=(16, 9), dpi=120)
```

Para exportar manteniendo esa calidad

```
plt.savefig('grafico_FullHD.png', dpi=120, bbox_inches='tight')
```

2. Calidad 4K (Ultra HD -> 3840 x 2160 píxeles)

Ideal para presentaciones en pantallas grandes o monitores de muy alta resolución.

Python

Mismo tamaño físico, pero el doble de densidad de píxeles

```
fig, ax = plt.subplots(figsize=(16, 9), dpi=240)
```

Para exportar

```
plt.savefig('grafico_4K.png', dpi=240, bbox_inches='tight')
```

3. Calidad "Publicación Científica" (Impresión máxima)

Si vas a imprimir el gráfico en una tesis, póster o revista, las dimensiones suelen ser más cuadradas o rectangulares (no de pantalla), pero la densidad de tinta debe ser extrema.

Python

Tamaño típico de página, densidad máxima (600 dpi es el estándar de imprenta alta)

```
fig, ax = plt.subplots(figsize=(8, 6), dpi=600)
```

```
plt.savefig('grafico_Paper.png', dpi=600, bbox_inches='tight')
```

El nivel Dios: "HD Infinito" (Vectores)

Si quieres la mejor calidad posible que **jamás se pixelee**, por más zoom que le hagas, olvídate de los píxeles (PNG/JPG) y exporta en formato **Vectorial**.

No importa qué figsize o dpi pongas, la gráfica se dibujará matemáticamente.

Python


```
fig, ax = plt.subplots(figsize=(10, 6))
```

```
# ... tu código del gráfico ...
```

```
# Exportar como SVG (Scalable Vector Graphics) o PDF
```

```
plt.savefig('grafico_infinito.svg', bbox_inches='tight')
```

```
plt.savefig('grafico_infinito.pdf', bbox_inches='tight')
```

- .svg: Es perfecto si quieres abrirlo en Adobe Illustrator o meterlo en una página web.
- .pdf: Es perfecto si quieres insertar la gráfica en un documento de LaTeX o Word sin perder calidad.

⚠ Una advertencia de memoria

Técnicamente, podrías poner `figsize=(100, 100)` y `dpi=1000` (lo que daría una imagen de 100,000 x 100,000 píxeles), pero **Python colapsará** por falta de memoria RAM al intentar dibujar el lienzo. Para gráficos basados en píxeles, el tope razonable suele rondar los `dpi=1000` en formatos de `figsize=(10, 8)`.

Pero quiero que se guarde con un tamaño más grande

Para que el gráfico se guarde con un tamaño físico más grande (más píxeles y que se vea imponente al abrirlo), debes aumentar los valores de `figsize` manteniendo un `dpi` alto.

Si solo subes el `dpi`, el gráfico se vuelve más nítido pero el tamaño relativo de las letras y líneas se mantiene igual. Si subes el `figsize`, estás agrandando el "lienzo" (las pulgadas).

Aquí tienes la configuración para generar un archivo **realmente grande** y de alta calidad:

1. El código para un tamaño "Gigante"

Para un gráfico que ocupe toda la pantalla en 4K o que sea apto para un póster grande:

Python

```
import matplotlib.pyplot as plt
```

```
# Configuramos un lienzo de 20x10 pulgadas (muy grande para Matplotlib)
```

```
# con un DPI de 200. Esto da una imagen de 4000 x 2000 píxeles.
```

```
fig, ax = plt.subplots(figsize=(20, 10), dpi=200)
```

```
# Graficamos
```

```
ax.plot(df_BC['dist_acum_m'], df_BC['P_BC'], label='Presión', linewidth=3) # Línea más gruesa para que no se vea delgada en el lienzo grande
```

IMPORTANTE: Al agrandar el figsize, debes agrandar las letras manualmente

```
ax.set_title('Perfil de Presiones', fontsize=25)
```

```
ax.set_xlabel('Recorrido [m]', fontsize=18)
```

```
ax.set_ylabel('Presión [kPa]', fontsize=18)
```

```
ax.tick_params(axis='both', labelsz=14) # Tamaño de los números en los ejes
```

```
ax.legend(fontsize=15)
```

```
ax.grid(True)
```

Guardar con el tamaño configurado

```
plt.savefig('grafico_gigante.png', bbox_inches='tight')
```

2. ¿Por qué se ve "pequeño" el texto si agrando el figsize?

Este es el error más común. Matplotlib usa puntos (pts) para las fuentes. Si el lienzo mide 5 pulgadas, una fuente de 12pts se ve bien. Pero si el lienzo mide 30 pulgadas, esa misma fuente de 12pts parecerá una hormiga.

Si vas a usar un figsize grande, sigue estas reglas:

- **Fuentes:** Súbelas a fontsize=20 o más.
- **Grosor de línea:** Usa linewidth=2 o 3 (el default es 1).
- **Marcadores:** Usa markersize=10 o más.

3. Comparativa de tamaños (en píxeles reales)

Configuración	Resolución Resultante	Uso ideal
figsize=(6, 4), dpi=100	600 x 400 px	Miniatura para web / WhatsApp.
figsize=(10, 6), dpi=150	1500 x 900 px	Estándar para informes de Word.

Configuración	Resolución Resultante	Uso ideal
figsize=(20, 10), dpi=200	4000 x 2000 px	Fondo de pantalla / Presentaciones HD.
figsize=(30, 15), dpi=300	9000 x 4500 px	Impresión de carteles o planos técnicos.

Un truco final: Si quieres que el archivo pese menos pero mantenga la calidad infinita (pudiendo imprimirlo del tamaño de un edificio sin que se pixele), usa: `plt.savefig('grafico.pdf', bbox_inches='tight')`

El PDF guarda las instrucciones matemáticas de las líneas en lugar de los píxeles individuales. Es el formato "más grande" posible técnicamente.

Cómo puedo conseguir los valores mínimos de una lista y los valores máximos? (primero globales y también locales)

Para encontrar estos valores en Python, la estrategia depende de si buscas los **globales** (el punto más alto o bajo de toda la lista) o los **locales** (los "picos" y "valles" intermedios).

Ya que estás trabajando con análisis de datos numéricos y gráficos, aquí tienes la forma más eficiente y profesional de hacerlo, usando las librerías estándar.

1. Valores Globales (El Máximo y Mínimo Absoluto)

Si solo necesitas el valor más grande y el más pequeño de toda tu lista o arreglo, Python y Pandas tienen funciones directas para esto.

En una lista normal de Python o un array de NumPy:

Python

```
import numpy as np
```

```
datos = [4, 1, 8, 3, 9, 2, 5]
```

```
# Obtener los valores
```

```
max_global = np.max(datos) # Devuelve 9
```

```
min_global = np.min(datos) # Devuelve 1
```

```
# Obtener la POSICIÓN (índice) donde están esos valores

indice_max = np.argmax(datos) # Devuelve 4 (la posición del número 9)

indice_min = np.argmin(datos) # Devuelve 1 (la posición del número 1)
```

En un DataFrame de Pandas:

Python

```
# Valor máximo y mínimo
```

```
max_df = df['Presion'].max()
```

```
min_df = df['Presion'].min()
```

```
# Fila exacta donde ocurre
```

```
indice_max_df = df['Presion'].idxmax()
```

2. Valores Locales (Picos y Valles)

Para encontrar los máximos y mínimos **locales**, necesitas identificar los puntos que son mayores (o menores) que sus vecinos inmediatos. Para esto, la herramienta definitiva es la librería **SciPy**, específicamente su módulo de señales.

El método a usar es `find_peaks` (encontrar picos).

El código paso a paso:

Python

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
from scipy.signal import find_peaks
```

```
# 1. Creamos unos datos de ejemplo que suben y bajan
```

```
y = np.array([0, 2, 5, 2, 1, 4, 7, 3, 1, 2, 0])
```

```
x = np.arange(len(y)) # El eje X (0, 1, 2, 3...)
```

```
# -----
```

```
# A. Encontrar MÁXIMOS LOCALES (Picos)
```

```
# -----
```

```

# find_peaks devuelve los índices donde están los picos
indices_max_locales, _ = find_peaks(y)

# -----

# B. Encontrar MÍNIMOS LOCALES (Valles)
# -----

# Truco matemático: find_peaks solo busca valores hacia arriba.
# Para encontrar los valles, multiplicamos la señal por -1 (la invertimos)
# y buscamos los picos de esa señal invertida.
indices_min_locales, _ = find_peaks(-y)

# --- VISUALIZACIÓN (Para que veas que funciona) ---
plt.figure(figsize=(8, 4))
plt.plot(x, y, color='blue', label='Datos', marker='o')

# Marcamos los máximos locales en rojo
plt.plot(x[indices_max_locales], y[indices_max_locales], "x", color='red', markersize=10,
label='Máximos Locales')

# Marcamos los mínimos locales en verde
plt.plot(x[indices_min_locales], y[indices_min_locales], "o", color='green', markersize=8,
label='Mínimos Locales')

plt.title('Detección de Máximos y Mínimos Locales')
plt.legend()
plt.grid(True)
plt.show()

```

¿Por qué find_peaks es tan poderoso?

En datos reales (como presiones o velocidades), a veces las señales tienen mucho "ruido" y falsos picos pequeños. La función `find_peaks` te permite filtrar eso fácilmente añadiendo parámetros:

- `prominence`: Te permite exigir que el pico destaque "X" unidades por encima de su entorno para ser considerado un pico real.
- `distance`: Exige que haya una distancia mínima en el eje X entre un pico y el siguiente (ideal si sabes que tus datos oscilan cada cierta distancia).
- `height`: Solo detecta picos que superen un valor Y específico.

Por ejemplo, si solo quieres picos que destaquen al menos 2 unidades de su entorno, usarías: `find_peaks(y, prominence=2)`.

Siguiendo algo similar a este formato:

```
# Valor máximo y mínimo
max_df = df['Presion'].max()
min_df = df['Presion'].min()
```

Cómo puedo calcular el promedio de esa columna?

Para seguir exactamente con la misma sintaxis limpia y directa de Pandas que estás usando, el método es `.mean()`.

Aquí tienes cómo calcular el promedio y un par de estadísticas que suelen ir de la mano en ingeniería:

1. Cálculo del Promedio

Python

```
# Valor promedio de la columna
promedio_df = df['Presion'].mean()
```

2. Otras estadísticas útiles (El "Combo" de Análisis)

Ya que estás analizando presiones, es muy probable que también necesites la **desviación estándar** (para saber cuánto varía la presión respecto al promedio) o la **mediana** (útil si tienes valores atípicos o "ruido" en las lecturas):

Python

```
# Mediana (el valor central de los datos)
mediana_df = df['Presion'].median()
```

Desviación estándar (qué tanto se alejan los datos del promedio)

```
std_df = df['Presion'].std()
```

Tip Pro: El comando "todo en uno"

Si quieres ver el máximo, mínimo, promedio y más de un solo golpe sin escribir línea por línea, puedes usar el método `.describe()`:

Python

```
# Genera un resumen estadístico completo de la columna
```

```
resumen = df['Presion'].describe()
```

```
print(resumen)
```

Esto te devolverá una tabla con:

- count: Cantidad de datos.
- mean: El promedio.
- std: Desviación estándar.
- min: Valor mínimo.
- 25%, 50% (Mediana), 75%: Percentiles.
- max: Valor máximo.

¿Necesitas filtrar los datos antes de promediar (por ejemplo, promediar solo cuando la bomba está encendida) o buscas el promedio global de todo el recorrido?

Es posible que me entregues un pdf con toda esta conversación?

Como IA, no tengo una función directa para "descargar" o generar un archivo PDF que se guarde automáticamente en tu computadora desde esta interfaz. Sin embargo, **puedes hacerlo tú mismo en menos de 10 segundos** de forma muy limpia.

Aquí tienes las dos mejores maneras de convertir nuestra conversación técnica en un manual de referencia PDF:

Método 1: El "Truco de Impresión" (El más rápido)

Es la forma más efectiva de mantener el formato del código y los gráficos.

1. Presiona Ctrl + P (en Windows) o Cmd + P (en Mac).
2. En la opción de "**Destino**" o "**Impresora**", selecciona "**Guardar como PDF**".

3. Asegúrate de que la opción "Gráficos de fondo" esté activada en la configuración para que se vean bien los bloques de código y colores.
4. Haz clic en **Guardar**.

Método 2: Copiar a una celda de Markdown (Para VS Code / Jupyter)

Si estás programando, lo ideal es tener esta guía en tu propio entorno:

1. Crea un archivo nuevo con extensión .md (ejemplo: Guia_Pandas_Fluidos.md).
2. Copia y pega las partes de esta conversación que más te sirvan.
3. Si usas VS Code, puedes instalar la extensión "**Markdown PDF**"; con un clic derecho sobre el archivo podrás convertirlo en un PDF con formato profesional.

Resumen de lo que hemos cubierto (por si quieres armar tu índice):

- **Lógica de df.where:** Para reemplazar datos que no cumplen condiciones.
- **Filtrado con loc e isin:** Para seleccionar escenarios como "Base" y "Marea alta".
- **Gráficos 3D:** Uso de projection='3d' y cómo unir puntos con líneas.
- **Gráficos 2D HD:** Comparación de múltiples presiones, personalización de colores, marcadores suaves (marker='.') y exportación a alta resolución (dpi).
- **Métodos Numéricos:** Implementación del **Método de Brent** (brentq) para resolver balances de energía implícitos.
- **Estadísticas en Pandas:** Uso de .max(), .min(), .mean() y .describe().

¿Hay algún otro cálculo de ingeniería o gráfico específico que quieras ajustar antes de que guardes tu guía?